
技术文档

智能仓储搬运系统

学校名称： 武汉理工大学

团队名称： 豆沙包队

队长： 姓名 彭乐恒

队员 1： 姓名 樊子诚

队员 2： 姓名 熊中本

2025 年 12 月

大模型技术创新赛 技术文档

AiC 智青春·算未来
2025全球校园人工智能算法精英大赛
The 7th Global Campus Artificial Intelligence Algorithm Elite Competition 2025

- 赛项：全球校园人工智能算法精英大赛·算法应用赛
- 作品名称：仓储智能搬运系统

全球校园人工智能算法精英大赛·算法应用赛·

大模型技术创新赛

智能仓储搬运系统技术文档

摘 要

本报告围绕“全球校园人工智能算法精英大赛·大模型技术创新赛”中的智能仓储分拣与搬运任务，基于 Ubuntu 22.04 操作系统与 ROS 2 (Humble) 机器人中间件，在 Gazebo 仿真环境中构建了一套高内聚、智能化的全自动作业系统。系统集成了大语言模型指令解析、全局任务规划、自主导航、机械臂运动控制及计算机视觉等前沿技术，旨在解决非结构化指令下的多目标物流调度难题。

本方案的核心创新在于设计了基于全排列的全局路径代价最优解算法（Global Path Cost Optimization based on Full Permutation）。针对复合型任务指令（如“抓取 3 个蓝色物块到 A 区，2 个红色物块到 B 区”），主控节点摒弃了传统的顺序执行逻辑，而是对所有待执行子任务生成全排列组合序列。系统通过实时计算机器人当前位姿、候选物块坐标及目标区域中心的三方欧氏距离，预演每种任务组合的行进总里程，从而筛选出总路径成本最低的全局最优执行顺序。该策略有效规避了“贪心算法”可能陷入的局部最优陷阱，在多目标、长距离的仓储场景中显著减少了无效往返移动，极大提升了作业效率。

整个系统以“指令解析→导航定位→路径规划→动态避障→货物识别→抓取搬运→放置反馈”为核心流程，通过主控节点模块间的高效通信与流程串联，实现从自然语言指令输入到货物精准放置的全自动化执行。

同时，方案兼顾场景复杂度提升与系统性能优化，满足竞赛对系统启动、指令解析、货物抓取放置、避障能力、状态反馈、场景设计及技术文档的全维度评分要求，确保任务稳定高效完成。

关键词：机械臂控制、自然语言指令解析、动态避障、最优抓取顺序

目 录

摘 要	2
第一章 项目概述	5
1.1 项目背景与目标	5
1.1.1 背景介绍	5
1.1.2 核心问题	5
1.1.3 项目目标	6
1.2 技术路线	7
1.2.1 任务分析	7
1.2.2 功能模块	9
1.2.3 程序流程	10
第二章 功能设计	14
2.1 系统总体架构	14
2.2 关键技术详解	15
2.2.1 路径规划算法	15
2.2.2 移动避障算法	17
2.2.3 机械臂规划算法	19
第三章 功能实现	21
3.1 模块实现细节	21
3.1.1 大语言模型实现	21
3.1.2 主控节点	22
3.1.3 Nav2 导航控制	24
3.1.4 视觉物块识别	25
3.1.5 MoveIt2 机械臂夹取控制	26
3.1.6 Foxglove 状态可视化	28

3.2 系统集成与调试	30
3.2.1 模块间通信方式	30
3.2.2 节点启动命令	33
3.2.3 避障参数调整	34
3.2.4 仿真物理环境鲁棒性优化	36
第四章 系统构建与导航节点解析	39
4.1 场景构建	39
4.1.1 基本场景构建	39
4.1.2 移动障碍物	39
4.1.3 Gazebo 仿真结果	40
4.2 系统实现完整流程	41
4.3 核心代码展示	43
参考文献	47

第一章 项目概述

1.1 项目背景与目标

1.1.1 背景介绍

大模型技术创新赛是以人工智能前沿技术为核心引擎的综合性赛事，聚焦大模型技术的突破性落地与生态体系搭建，旨在激活高校团队在自然语言处理、多模态交互、知识推理等领域探索大模型创新应用的潜能。赛事开放真实场景数据集与算力支持，激励参赛者突破模型轻量化、推理效率提升、垂直场景适配等核心技术瓶颈，倡导打造可持续发展的 AI 创新生态。赛事不仅为技术人才搭建展示与交流的优质平台，更着力打通产学研用一体化链条，推动大模型技术从实验室研发向产业赋能的高效转化，为数字经济时代的智能化升级注入动力。

为促进大模型技术在智能物流领域的创新性落地，本赛题聚焦仓储基地分拣与搬运核心场景，参赛团队需依托指定设备，融合大模型、计算机视觉、自主导航、机械臂抓取等技术，设计一套智能作业系统，攻克货物识别定位、自主导航避障、精准抓取放置等核心技术难题。本赛项旨在激发参赛团队的技术创新活力，促进智能物流技术与可持续发展理念的深度融合。

1.1.2 核心问题

（1）自然语言指令的精准解析与系统适配

如何从裁判发布的自然语言任务中，准确提取货物颜色（红 / 蓝）、数量（整数）、目标区域（A/B/C 区）等关键参数，转化为机器可执行的结构化指令；如何实现解析结果与 ROS2 系统的接口兼容，确保指令能被导航、机械臂等模块实时响应。

（2）静态环境下的最优路径规划与避障

如何让移动机器人规划从初始位置到货物区域的最优路径，平衡路径长度与移动

效率；如何基于激光雷达数据精准识别仓储固定结构、物块等静态障碍物，确保导航过程中无碰撞；如何保证机器人抵达目标货物位置的定位精度，为后续抓取动作提供基础。

（3） 仓储环境中目标货物的实时识别

如何基于相机传感器与 OpenCV 技术，在仿真环境可能存在的光照变化、背景干扰下，实时识别红 / 蓝颜色包裹；如何过滤小面积噪声或非目标物体干扰，提升颜色识别的鲁棒性；如何确保识别速度满足实时性要求，不影响整体任务流程。

（4） 机械臂的精准抓取与放置协同控制

如何通过 MoveIt2 框架实现机械臂的无碰撞轨迹规划与逆运动学求解，精准抓取目标货物；如何保证抓取后货物在搬运过程中稳定无脱落；如何实现抓取与导航的动作衔接，确保机器人抵达放置区域后，机械臂能精准释放货物至指定位置。

（5） 全流程状态的实时可视化与数据同步

核心挑战：如何通过 Rosbridge 协议实现 ROS2 话题数据向 Foxglove 的实时传输；如何自定义 Foxglove 可视化面板，清晰展示指令解析结果、导航轨迹、抓取状态及任务进度；如何确保可视化信息与机器人实际动作同步，避免“显示与执行脱节”，满足任务进度可监控的需求。

1.1.3 项目目标

本项目聚焦大模型技术与智能机器人技术的深度融合，以仓储分拣搬运场景为核心，基于 Gazebo+ROS2 仿真环境构建兼具创新性、高效性与鲁棒性的智能作业系统，核心目标如下：

（1） 指令解析智能化：

突破传统指令转换的局限性，实现自然语言任务到机器人指令的“理解-分解-适配”全流程智能处理，探索大模型在机器人自主作业中的轻量化落地路径。

（2） 导航避障决策化：

摒弃固定路径规划模式，构建“感知-决策-优化”闭环导航系统，既能精准识别静

态障碍物与动态干扰，又能基于任务优先级动态调整最优路径，在保证安全无碰撞的前提下最大化作业效率，适配 5 分钟限时竞赛要求，为动态仓储环境提供智能导航新范式。

（3）系统鲁棒化：

融合多传感器数据与导航算法策略，提升复杂场景下的运行可靠性，实现仿真环境传感器噪声、动态障碍干扰、动作执行偏差下的核心导航任务稳定完成，探索结构化场景中移动机器人系统的高效优化路径。

（4）抓取放置协同化：

实现机械臂运动规划与机器人导航的深度协同，优化抓取轨迹与放置策略，既保证货物抓取的稳定性（无脱落、无弹飞），又实现放置位置的精准控制，解决“抓取-搬运-放置”全流程动作衔接的核心难题，提升机器人作业的自动化水平。

（5）状态可视化交互化：

创新 Foxglove 可视化模式，实时展示指令解析结果、导航轨迹、抓取状态等核心信息，支持通过可视化面板实时监控作业进度、快速定位问题，为机器人自主作业的调试与优化提供直观高效的支撑，探索机器人作业可视化的新形态。

（6）系统集成可扩展化：

实现各功能模块的高效协同与稳定运行，在 5 分钟竞赛限时内完成指定任务的同时，构建模块化、可复用的技术架构，既满足竞赛场景的严苛要求，又为后续实机部署、复杂仓储场景扩展（如多机器人协同、多类型货物分拣）铺垫基础，为智能物流领域提供轻量化、高适配的技术解决方案。

1.2 技术路线

1.2.1 任务分析

（1）大语言模型指令解析：

核心目标是实现一个基于 ROS 2 的自然语言命令解析节点，将用户输入的

自然语言指令（如“抓取 1 个蓝色物块去 B”）通过大模型 LLaMA.cpp 服务端解析为机器人可执行的结构化任务数据（包含颜色、数量、目标区域），并通过 ROS 2 话题发布，同时解决命令冲突（确保仅处理最新命令），为机器人任务调度提供标准化输入。

(2) 主控节点 (main_controller_node) :

是整个机器人系统的任务调度与逻辑控制中心，负责接收用户指令、规划任务执行流程、协调导航与机械臂动作，实现物块抓取 - 放置的全流程自动化。

其核心目标包括：

- 1) 将大模型解析节点发布的话题（如“将 2 个红色物块放到 A 区”），转化成可执行的任务序列。
- 2) 优化任务规划最优任务执行顺序，通过计算路径成本（距离总和）最小化总移动距离，提升效率。
- 3) 实时监控机器人位置、障碍物动态和任务执行状态，实现动态避障与流程容错。
- 4) 协同导航系统、机械臂控制系统和物体检测系统 (simple_navigator) (arm_grab_place_node) (color_detector_node), 确保各模块按序协作。

(3) Nav2 导航控制:

核心目标是接收目标位置指令，通过路径规划、运动控制和避障算法，使机器人安全、准确地到达指定位置，同时与主控节点 (main_controller) 形成闭环协作，完成复杂任务（如物块抓取与搬运）。具体来说，它要解析并执行来自主控节点的导航目标，这些目标可以是预设区域或自定义坐标；实时反馈导航状态，包括成功、失败或取消，供主控节点决策下一步动作决策；还要结合地图与传感器数据，实现自主避障和路径优化。

(4) MOVEIT2 机械臂夹取控制:

基于 MoveIt2 框架实现机械臂与夹爪的精准姿态控制，通过“预抓取→抓取→举高→放置→复位”的标准化动作序列完成货物夹取与释放，并通过 ROS 2 通信机

制与主控节点实时联动，确保夹取动作与导航流程（到达抓取点 / 目标区域）无缝衔接，最终实现货物稳定转移。核心在于姿态的精确控制（如机械臂弯曲角度、夹爪开合程度）和与系统中枢的状态同步。

（5）状态实时可视化：

在任务执行全过程中，需将关键信息（包括指令解析结果、机器人导航轨迹、机械臂抓取状态、每轮任务完成情况）实时显示在 Foxglove 工具中，确保任务进度可监控。

1.2.2 功能模块

本智能仓储搬运系统，以主控节点作为系统核心，进行任务的分配与调度，以下是各功能模块与主控节点的交互表 1-1.

交互模块	通信接口类型	话题/服务名称	功能描述
导航节点 (simple_navigator)	发布话题	/manual_nav_target	向导航节点发送目标位置 (JSON 格式，支持预设区域或自定义坐标)，触发导航动作。
	订阅话题	/nav_status	接收导航状态反馈 (succeeded/failed)，判断是否到达目标点。
机械臂控制节点 (arm_grab_place_node)	发布话题	/current_target_cube	通知机械臂目标物块名称 (如 “red_cube_1”)，用于吸附 / 分离动作。
	发布话题	/nav_done_cargo	通知机械臂已到达物块附近，触发抓取流程。
	发布话题	/nav_done_area	通知机械臂已到达目标区域，触发放置流。
	订阅话题	/arm_status	接收机械臂动作反馈 (如抓取 / 放置完成)，推进任务流程。
指令解析节点 (llama_command_parser)	订阅话题	/chat	接收解析后的任务指令 (JSON 格式，含颜色、数量、目标区域)，生成任务队列。

交互模块	通信接口类型	话题/服务名称	功能描述
定位系统（AMCL）	订阅话题	/amcl_pose	接收机器人在地图中的实时位姿（x, y 坐标），用于路径规划和距离计算。
物体检测节点 （color_detector_node）	间接依赖	-	通过预设物块位置（RED_BLOCKS/BLUE_BLOCKS）关联检测结果，辅助目标选择。

表 1-1 各功能模块与主控节点的交互表

1.2.3 程序流程

1.程序流程图：

整个系统以 “指令解析→导航定位→路径规划→动态避障→货物识别→抓取搬运→放置反馈” 为核心流程，通过主控节点模块间的高效通信与流程串联，实现从自然语言指令输入到货物精准放置的全自动化执行。具体流程如图 1-1：

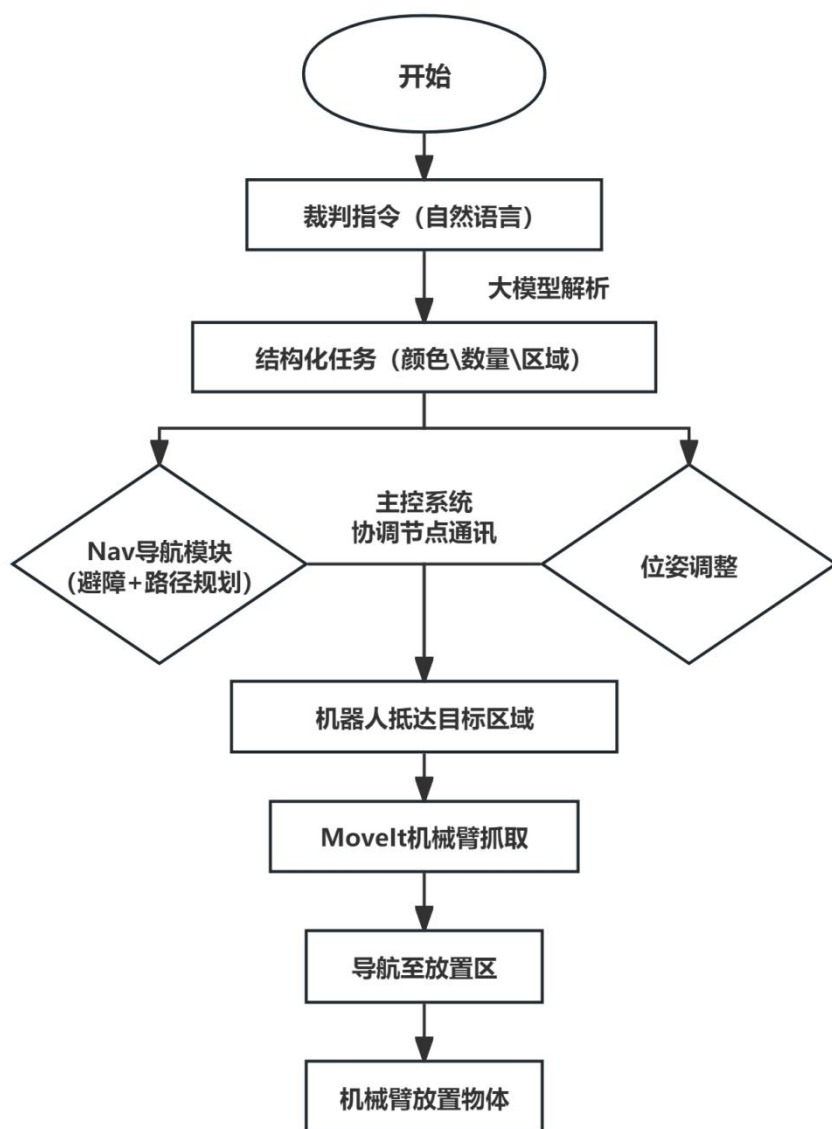


图 1-1 程序流程图

2.系统层级关系:

基于代码库的模块划分，系统采用四层架构设计，各层级职责与核心模块对应关系如表 1-2:

层级	核心功能	涉及模块	技术支撑
----	------	------	------

交互层	自然语言命令解析与可视化反馈	llama_command_parser	LLaMA 大模型、Foxglove 面板
决策层	任务规划与动态调度	main_controller	路径优化算法、避障逻辑
执行层	机械臂操作与导航控制	arm_action_integration nav_simple、yzbot	ROS2 动作服务、Gazebo 仿真
感知层	物块识别与环境感知	package_detector auto_move_obstacle	计算机视觉、传感器数据融合

表 1-2 关键数据传输路径表

3.系统数据流图

系统数据流主要包含三大部分，分别是命令输入流、任务执行流、感知数据流，具体见图 1-2

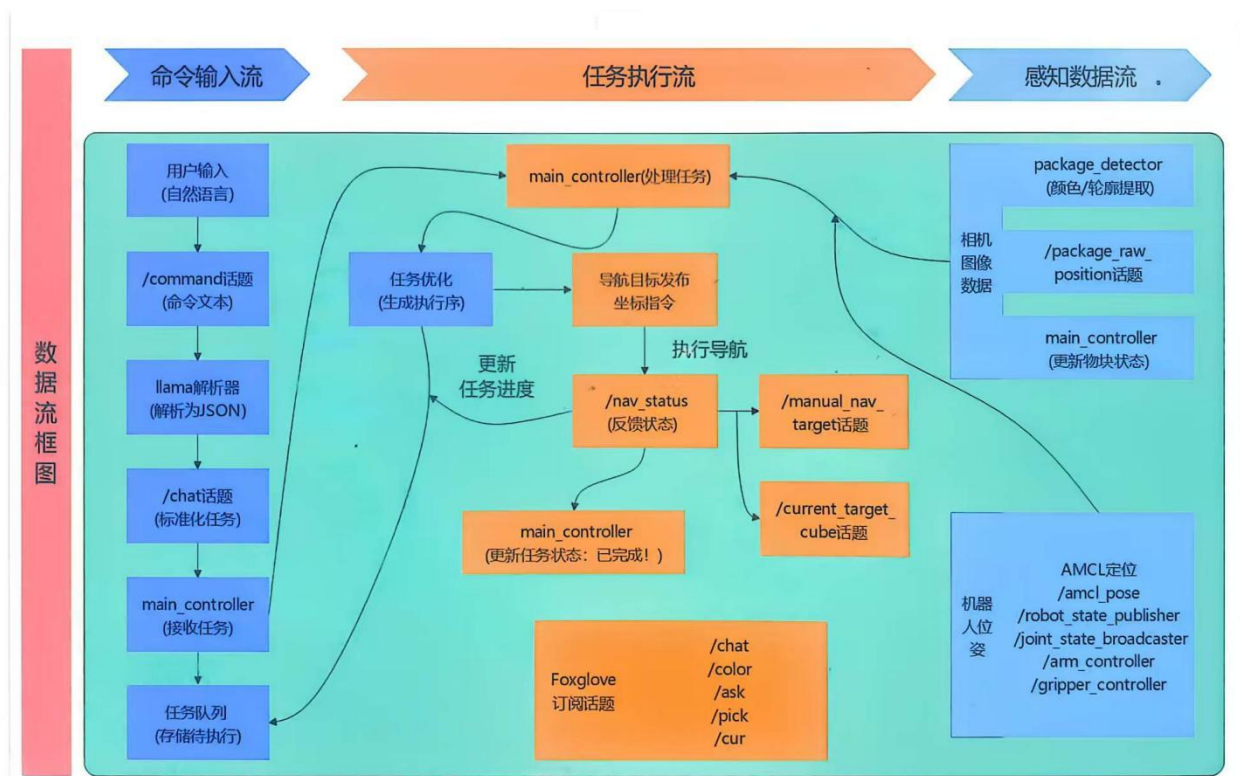


图 1-2 数据流示意图

第二章 功能设计

2.1 系统总体架构

系统总体框架如图 2-1

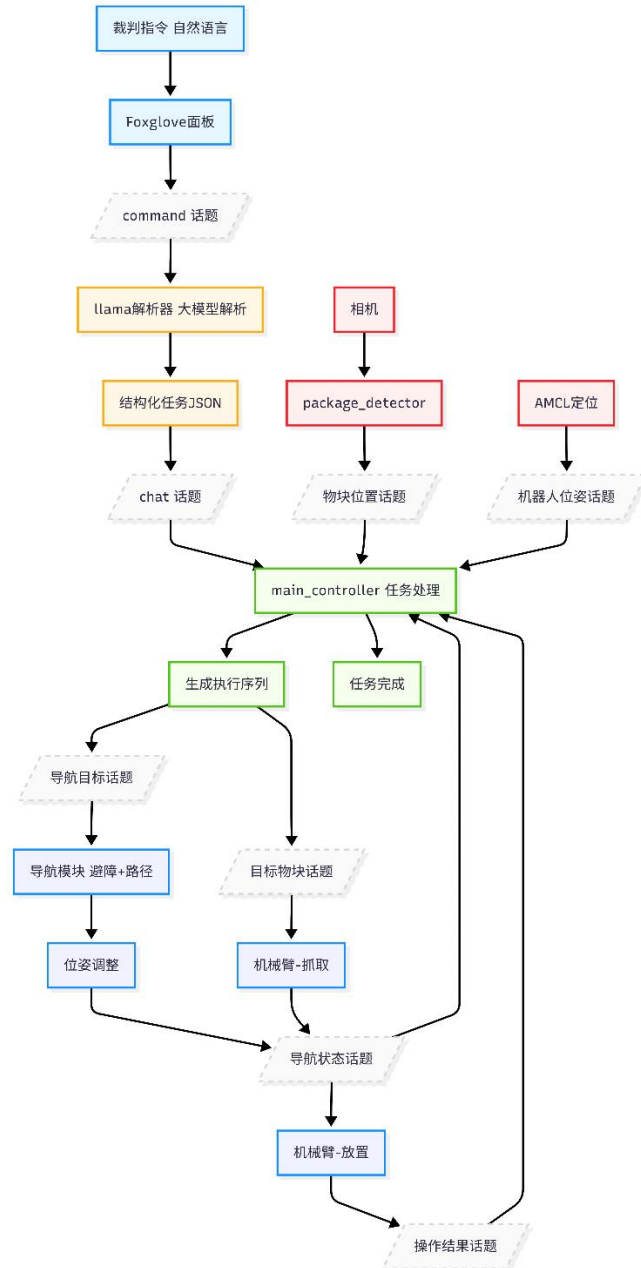


图 2-1 系统总体架构图

2.2 关键技术详解

2.2.1 路径规划算法

本系统的路径规划模块采用基于全排列的全局路径代价最优解算法（Global Path Cost Optimization based on Full Permutation），旨在解决非结构化指令下的多目标物流调度难题。

算法首先通过自然语言解析模块将高层指令（如“抓取两个蓝色去 B 区”）转化为原子化的结构化任务数据。针对多任务协同场景，系统默认优先采用全排列策略构建解空间：通过遍历所有待执行任务的排列组合，实时计算机器人当前位姿、候选物块坐标及目标区域中心的三方欧氏距离，预演每种序列的行进总里程，从而筛选出总路径成本最低的全局最优执行顺序。该策略有效规避了传统逻辑易陷入局部最优的缺陷，显著减少了无效往返移动。具体代码如图 2-2：

```
1     def calculate_task_sequence_cost(self, tasks):
2         total_cost = 0.0
3         current_pos = self.current_robot_pose
4         # 计算从当前位置到每个任务的抓取点再到目标区域的总距离
5         for assignment in task_assignments:
6             total_cost += self.calculate_distance(current_pos,
7 assignment["grasp_pos"])
8             total_cost +=
9 self.calculate_distance(assignment["grasp_pos"],
assignment["area_pos"])
            current_pos = assignment["area_pos"]
        return total_cost
```

图 2-2 物块动态选择机制代码图

为兼顾算法的理论最优性与工程实时性，系统引入了自适应分级优化机制。在常规规模的任务场景中，坚持全局搜索以确保效率最大化；仅在单任务涉及的物块组合规模超出计算阈值（如组合数 > 1000）时，算法才会自动降级为启发式贪心策略（Heuristic Greedy Strategy）。此时，系统将优先选择距离当前位姿最近的可行目标，

以在毫秒级时间内给出次优解，确保主控节点在高负载下仍能保持高频响应。具体代码如图 2-3:

```
01 def calculate_task_sequence_cost(self, tasks):
02     total_cost = 0.0
03     current_pos = self.current_robot_pose # 起始位置为机器人当前位置
04     used_blocks = set() # 记录已分配物块，避免重复使用
05     task_assignments = []
06
07     # 为每个任务分配最佳物块
08     for task in tasks:
09         best_block, cost = self.assign_best_block_to_task(task, used_blocks)
10         if not best_block:
11             return float("inf") # 物块不足，返回无穷大成本
12
13         block_x, block_y, block_idx = best_block
14         target_area = task["to"]
15         area_pos = self.AREA_COORDS[target_area]
16         grasp_pos = self.get_grasp_position((block_x, block_y))
17
18         # 记录分配结果
19         task_assignments.append({
20             "task": task,
21             "block_pos": (block_x, block_y),
22             "block_idx": block_idx,
23             "grasp_pos": grasp_pos,
24             "area_pos": area_pos
25         })
26         used_blocks.add(block_idx)
```

图 2-3 自适应策略算法代码

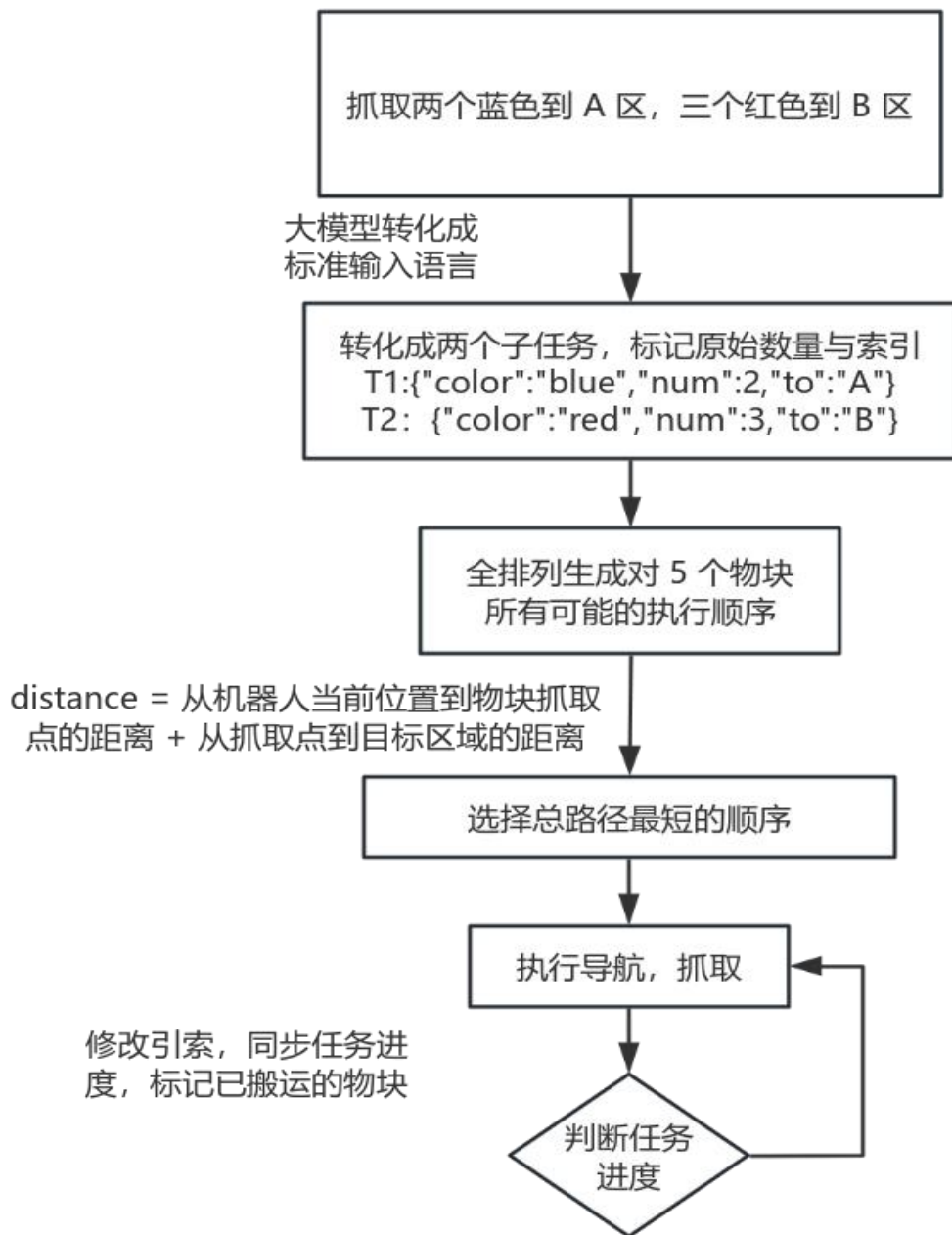


图 2-4 路径规划算法流程图

2.2.2 移动避障算法

移动避障算法的核心逻辑是“实时环境感知 - 碰撞风险量化 - 动态决策执行”

的闭环控制，通过机器人对自身与障碍物状态的精准掌握，结合预设规则实现安全路径规划。机器人需实时同步两个关键信息以支撑避障决策：一方面通过订阅/amcl_pose 话题获取自身实时位姿（current_robot_pose），确保对自身位置的精准认知；另一方面借助定时器（obstacle_timer，间隔 OBSTACLE_CHECK_INTERVAL = 0.5 秒）周期性更新障碍物的实时坐标（current_pos），无需关注障碍物的移动细节，仅需获取其当前位置即可满足判断需求。

碰撞风险的量化是避障逻辑的核心环节，通过预设安全距离与实时距离计算，将环境风险转化为可执行的决策信号。系统设定 SAFE_DISTANCE = 1.5 米作为临界值（包含机器人自身尺寸与缓冲空间），并通过欧氏距离公式实时计算机器人与障碍物的直线距离，核心实现为 def calculate_distance(self, point1, point2): return math.hypot(point1[0] - point2[0], point1[1] - point2[1])，该函数直接输出两点间距离，为风险判断提供量化依据。具体代码如图 2-5

```
1 # 简化的风险判断逻辑
2 for obs_info in self.MOVING_OBSTACLES.values():
3     dist = self.calculate_distance(self.current_robot_pose,
4     obs_info["current_pos"])
5     if dist < self.SAFE_DISTANCE:
6         self.current_step = "WAIT_OBSTACLE" # 进入避障等待状态
7         self.wait_start_time = time.time() # 记录等待开始时间
        break
```

图 2-5 碰撞风险量化代码

当检测到碰撞风险时，系统通过状态机切换实现行为调整，在保障安全的同时兼顾任务连续性。若机器人与障碍物的距离小于 SAFE_DISTANCE，主控制器会将当前状态（current_step）切换为 WAIT_OBSTACLE，暂停当前导航任务并启动等待定时器（wait_timer），核心逻辑为遍历障碍物信息计算距离，一旦触发阈值则更新状态并记录等待开始时间。等待期间，系统每 0.5 秒重新检查距离，若距离恢复至安全范围（≥SAFE_DISTANCE），则切换回正常任务状态（如 NAV_TO_BLOCK）；若等待超时（超过 MAX_WAIT_TIME = 30 秒），则触发路径重规划或其他备选策略。

移动避障的核心在于“量化风险 + 状态闭环”，通过实时同步自身与障碍物位置

实现环境感知，基于安全距离阈值完成风险量化，再通过状态切换触发等待或恢复任务的决策，最终在动态环境下实现安全移动。这一逻辑通过 `MainController` 中的距离计算、状态管理与定时器机制实现，无需依赖复杂的障碍物运动模型，仅通过“位置 - 距离 - 决策”的简单闭环即可高效完成避障。

2.2.3 机械臂规划算法

机械臂的夹取放置功能是一个多环节协同的自动化过程，核心围绕目标识别、动作控制与任务调度的有机结合展开。

目标识别与定位是整个流程的起点。系统通过视觉感知技术，先对相机捕捉的图像进行处理，依据颜色特征区分不同目标物块，同时过滤掉背景干扰和噪声。对于识别到的物块，会提取其轮廓并确定中心位置，再结合相机的参数信息，将图像中的二维坐标转换为实际空间中的三维坐标，以精准定位物块位置。为确保识别的准确性，还会结合其他传感数据对目标的真实性进行验证，避免误判。

机械臂夹取放置的前提是明确目标物块，系统通过订阅 `/current_target_cube` 话题实时更新目标信息。在 `ArmGrabPlaceNode` 中，`target_cube_callback` 函数会校验物块名称（需包含“`cube`”且以“`red_`”或“`blue_`”开头），并更新吸附参数中的 `model2_name`，确保后续动作针对正确目标。具体代码如图 2-6

```
1 def target_cube_callback(self, msg):
2     new_cube_name = msg.data.strip()
3     if "_cube_" in new_cube_name and (new_cube_name.startswith("red_") or
4     new_cube_name.startswith("blue_")):
5         self.attach_params["model2_name"] = new_cube_name # 更新目标物块名称
```

图 2-6 目标物块识别代码

动作序列规划是实现夹取与放置的关键。机械臂需要按照预设的轨迹调整姿态，从初始状态平稳过渡到适合抓取的位置。随后，夹爪根据指令完成开合动作，在抓取时实现对物块的稳固夹持，放置时则平稳释放。为了在仿真环境中模拟物块与机械臂的物理连接，系统会通过虚拟吸附机制，在抓取时建立机械臂末端与物块的固定关系，

确保移动过程中物块不会脱落，放置时再解除这种连接。

(1) 机械臂姿态调整：

通过 `send_arm_action` 函数发送关节轨迹，将机械臂从初始姿态 (`arm_trajectory["init"]`) 调整至抓取位姿 (如 "bend" 姿态: `[0.0, 1.2, 1.17, 0.0, -0.3, 0.0]`)。轨迹包含起始点与目标点，通过 `JointTrajectoryPoint` 定义位置与时间参数，确保动作平滑过渡：

(2) 夹爪开合控制：

`send_gripper_action` 函数根据 "close" 或 "open" 指令，通过 `gripper_trajectory` 发送夹爪关节轨迹 (如闭合动作包含 `[0.0, 0.1, 0.2]` 位置序列)，时间步长优化为 2 秒内完成，提升效率。

(3) 虚拟吸附 / 分离：

当机械臂与夹爪到位后，调用 `send_attach_action` 或 `send_detach_action` 服务，通过 `AttachLink.Request` 参数指定机械臂模型 (`six_arm`)、末端连杆 (`link6`)、物块模型 (如 `red_cube_1`) 及物块连杆 (`link`)，在 Gazebo 中建立或解除固定关节，实现抓取与释放。

(4) 状态反馈：

每个动作完成后，通过 `/arm_status` 发布者发送确认信号 (如 "grasp_done" 或 "place_done")，为主控节点 (`MainController`) 提供执行状态，确保任务流程同步。

整个过程离不开状态反馈与主控系统的协同调度。机械臂会实时向主控系统反馈当前的执行状态，如抓取完成、放置完成等，主控系统则根据这些状态触发下一步操作，比如在抓取完成后指挥机器人导航至放置区域。同时，系统还具备容错机制，若抓取失败会自动重试，若超时未完成任务则启动异常处理。主控系统还会根据物块位置、目标区域和机器人当前状态，优化多任务的执行顺序，减少无效移动，提高整体效率。

第三章 功能实现

3.1 模块实现细节

3.1.1 大语言模型实现

(1) 节点初始化

- 1) 创建 ROS 2 节点 `llama_command_parser`，配置 LLaMA 服务地址与模型参数（输出长度、随机性等）。
- 2) 初始化任务管理机制：任务计数器（生成唯一 ID）、活跃任务 ID（标记最新命令）、处理锁（避免并发冲突）。
- 3) 建立 ROS 通信：创建 `/chat` 话题发布者（输出解析结果）、`/command` 话题订阅者（接收自然语言命令）。

(2) 命令接收与预处理

- 1) 提取命令文本（兼容 JSON 封装或纯文本），过滤空命令或不含关键信息（颜色 / 区域）的无效命令。
- 2) 生成新任务 ID 并标记为“活跃”，终止旧任务（通过状态锁确保仅处理最新命令），触发解析流程。

(3) 命令解析与发布

- 1) 构造提示词：严格约束 LLaMA 输出格式（仅含 `color/num/to` 的 JSON 数组），附示例引导模型输出。
- 2) 调用大模型：向 LLaMA 服务发送请求，获取解析结果。
- 3) 清洗与解析：通过多层容错（基础清洗→正则提取→JSON 解析）处理模型输出，确保结构化数据有效。
- 4) 校验与发布：仅发布当前活跃任务的结果到 `/chat` 话题，完成后释放处理锁。

(4) 辅助核心：JSON 容错解析 (clean_and_parse_json)

清洗模型输出中的多余符号(如代码块标记、标点),通过正则提取 JSON 数组,双重解析确保格式正确。

3.1.2 主控节点

3.1.2.1 节点初始化与核心配置

(1) 初始化预设参数:

- 1) 物块位置: 存储红 / 蓝色物块初始坐标及抓取状态
- 2) 目标区域: 定义 A/B/C 区坐标
- 3) 移动障碍物: 配置运动参数(起点、终点、速度等)
- 4) 避障与超时参数: 安全距离 (SAFE_DISTANCE = 0.5m)

(2) 建立 ROS 通信机制:

- 1) 订阅器: /amcl_pose (机器人位姿)、/chat (任务指令)、/nav_status (导航状态)、/arm_status (机械臂状态)
- 2) 发布者: /current_target_cube (目标物块)、/manual_nav_target (导航目标)、/nav_done_cargo (抓取触发)、/nav_done_area (放置触发)
- 3) 启动定时器: 障碍物位置更新定时器(间隔 0.5s, 调用 _update_obstacle_positions)

3.1.2.2 任务接收与解析处理

接收任务指令: 通过/chat 话题订阅来自 llama_command_parser 的 JSON 格式任务列表(如[{"color": "red", "num": 2, "to": "A"}])。将解析后的任务存入 task_queue, 每个任务包含颜色、数量和目标区域

- 1) 初始化状态变量: current_task (当前任务)、completed_num (已完成数量) 等
- 2) 任务优化排序:

调用 `optimize_task_order` 函数生成最优执行顺序

通过全排列算法（`permutations`）计算所有可能的任务排列组合

选择总路径成本最低的序列作为 `optimized_tasks`

3) 任务执行与状态控制

状态机设计：核心状态包括 `WAIT_TASK`（等待任务）、`NAV_TO_BLOCK`（导航到物块）、`GRASP`（抓取）、`NAV_TO_AREA`（导航到区域）、`PLACE`（放置）、`WAIT_OBSTACLE`（等待障碍物），通过 `current_step` 变量跟踪。

`_grasp_position` 计算物块偏移位置（考虑机械臂操作范围）。

单个任务完成后从 `optimized_tasks` 移除，继续执行下一任务，所有任务完成后回到 `WAIT_TASK` 状态，等待新指令。

4) 核心辅助功能

距离计算：`calculate_distance` 函数通过两点坐标计算欧式距离

物块分配：`assign_best_block_to_task` 为任务选择最优物块

成本评估：`calculate_task_sequence_cost` 计算任务序列总路径成本（含抓取到放置全流程）

5) 节点运行

启动节点后初始化所有参数与通信接口，进入 `WAIT_TASK` 状态接收任务后，自动触发优化与执行流程，实时更新状态并处理异常，通过日志输出关键节点信息（任务进度、导航状态、避障事件等），如图 3-1



图 3-1 主控节点启动图

3.1.3 Nav2 导航控制

(1) 导航节点初始化 (SimpleNav2Navigator)

具体如图 3-2

```
01     class SimpleNav2Navigator(Node):
02         def __init__(self):
03             super().__init__("simple_nav2_navigator")
04             # 初始化 Nav2 动作客户端 (对接/navigate_to_pose 接口)
05             self.nav_client = ActionClient(self, NavigateToPose,
06             "/navigate_to_pose")
07             # 订阅主控节点发送的导航指令 (/manual_nav_target 话题)
08             self.target_sub = self.create_subscription(
09                 String,    "/manual_nav_target",
10             self.target_callback, 10
```

```

11         )
12         # 发布导航状态给主控节点 (/nav_status 话题)
13         self.status_pub = self.create_publisher(String,
"/nav_status", 10)

        # 预设区域坐标 (与主控节点的 AREA_COORDS 对应)

        self.area_coords = {"A": (2.0, 1.0, 0.0), "B": (0.0, 3.0,
1.57), "C": (-2.0, 1.0, 3.14)}

```

图 3-2 导航节点初始化代码

(2) 与主控节点的指令交互

接收导航目标：主控节点通过/manual_nav_target 话题发送 JSON 格式指令，包含目标类型（预设区域或自定义坐标）：

发送导航状态：导航节点通过/nav_status 话题向主控节点反馈结果（成功 / 失败 / 取消），供主控节点切换任务状态（如从 “导航到物块” 转为 “抓取”）：

(3) 导航目标执行流程

构造目标位姿：将接收的坐标 (x, y, yaw) 转换为 PoseStamped 消息（基于地图坐标系）：

路径规划：由 nav2_planner 根据地图生成从当前位置到目标的路径。

运动控制：nav2_controller（如 MPPIController）根据路径生成速度指令，控制机器人移动。

避障：结合传感器数据，通过 collision_monitor 模块实时避开障碍物。

结果反馈：导航完成后，通过 result_callback 处理结果，并将状态发布给主控节点。

3.1.4 视觉物块识别

(1) 传感器数据获取与预处理

-
- 1) 视觉模块：通过相机订阅 RGB 图像，转换为 HSV 色彩空间，利用预设的红、蓝两种颜色的 HSV 阈值范围生成颜色掩码，分割出对应颜色的区域；再通过轮廓提取（cv2.findContours）筛选出面积符合条件的物块轮廓，仅识别并记录物块的颜色信息。）
 - 2) 雷达模块：订阅激光雷达数据（/scan），解析为二维空间点（x, y），用于验证视觉检测到的颜色区域是否对应真实物理物体。

(2) 数据融合与存在性验证颜色与存在性验证：

视觉模块确定物块颜色后，结合雷达数据验证该颜色区域是否对应真实物体——通过对比视觉轮廓中心的方向向量与雷达点的方向向量，判断两者是否指向同一空间区域，以此确认颜色区域的真实性。

(3) 结果发布与状态反馈发布检测到的物块颜色信息及相关状态；

实时发布节点状态（/color_detector/status），包括是否检测到物体、传感器数据接收情况等，便于系统监控与调试。

3.1.5 MoveIt2 机械臂夹取控制

(1) 初始化与通信链路搭建

机械臂与夹爪动作客户端初始化：

创建 FollowJointTrajectory 动作客户端，分别连接 arm_controller 和 gripper_controller，等待服务就绪（如 arm_action_client.wait_for_server()）；

吸附 / 分离服务客户端初始化：

创建 AttachLink/DetachLink 服务客户端，绑定

/ATTACHLINK//DETACHLINK 服务，用于物理吸附控制；

通信接口配置：

订阅主控节点的触发话题（/nav_done_cargo//nav_done_area）和目标物块话题（/current_target_cube），创建/arm_status 发布者用于状态反馈。

(2) 姿态规划与轨迹定义

机械臂关键姿态预设：

通过关节角度数组定义核心姿态，如：`init`（初始姿态）：`[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]`（关节 1-6 复位）；`bend`（抓取准备姿态）：`[0.0, 1.2, 1.17, 0.0, -0.3, 0.0]`（手臂弯曲至货物高度）；`lift`（举高姿态）：`[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]`（抓取后提升货物避免碰撞）。

夹爪动作轨迹定义：

通过位置序列控制开合程度，如：`close`（闭合）：`[0.0, 0.1, 0.2]`（逐步收紧夹爪）；`open`（打开）：`[0.3, 0.2, 0.1, 0.0]`（逐步松开）。

（3）夹取 / 放置动作流程（与主控联动）

抓取流程（响应/`nav_done_cargo` 触发）：

机械臂从初始姿态切换至 `bend` 姿态（贴近货物），通过 `send_arm_action` 发送轨迹，确保末端执行器对准货物；夹爪执行 `close` 动作（逐步闭合），通过 `send_gripper_action` 控制 `finger_joint1` 位置，夹紧货物；调用/`ATTACHLINK` 服务，将机械臂末端连杆（`link6`）与货物（如 `red_cube_1`）吸附，物理固定；机械臂切换至 `lift` 姿态（举高货物），避免移动时碰撞；完成后通过/`arm_status` 发布 “`grasp_done`”，通知主控节点可开始导航至目标区域。

放置流程（响应/`nav_done_area` 触发）：

机械臂从举高姿态切换至 `bend` 姿态（降低货物至放置高度）；调用/`DETACHLINK` 服务，解除机械臂与货物的吸附；夹爪执行 `open` 动作（松开货物）；机械臂复位至 `init` 姿态，准备下一次动作；完成后通过/`arm_status` 发布 “`place_done`”，通知主控节点当前任务完成。

（4）容错与状态同步

动作回调机制：

每个动作（机械臂 / 夹爪 / 吸附）执行后通过回调函数（如 `_arm_result_done_cb`）检查结果，失败则触发 `_reset_execution` 重置状态，避免流程阻塞；

与主控的状态闭环：

主控节点通过监听/arm_status 判断动作是否完成，若超时（如 GRASP_TIMEOUT=8 秒）则触发重试逻辑。

3.1.6 Foxglove 状态可视化

订阅话题与发布者如表 3-1

话题	发布者	用途
/chat	llama_command_parser_node	发布经大模型解析后的任务指令
/color	main_controller_node	发布目标物块颜色对应的标识
/ask	main_controller_node	发布与目标颜色对应的任务标识
/pick	main_controller_node	发布抓取状态标识
/cur	main_controller_node	发布作业状态标识

表 3-1 订阅话题与发布者表

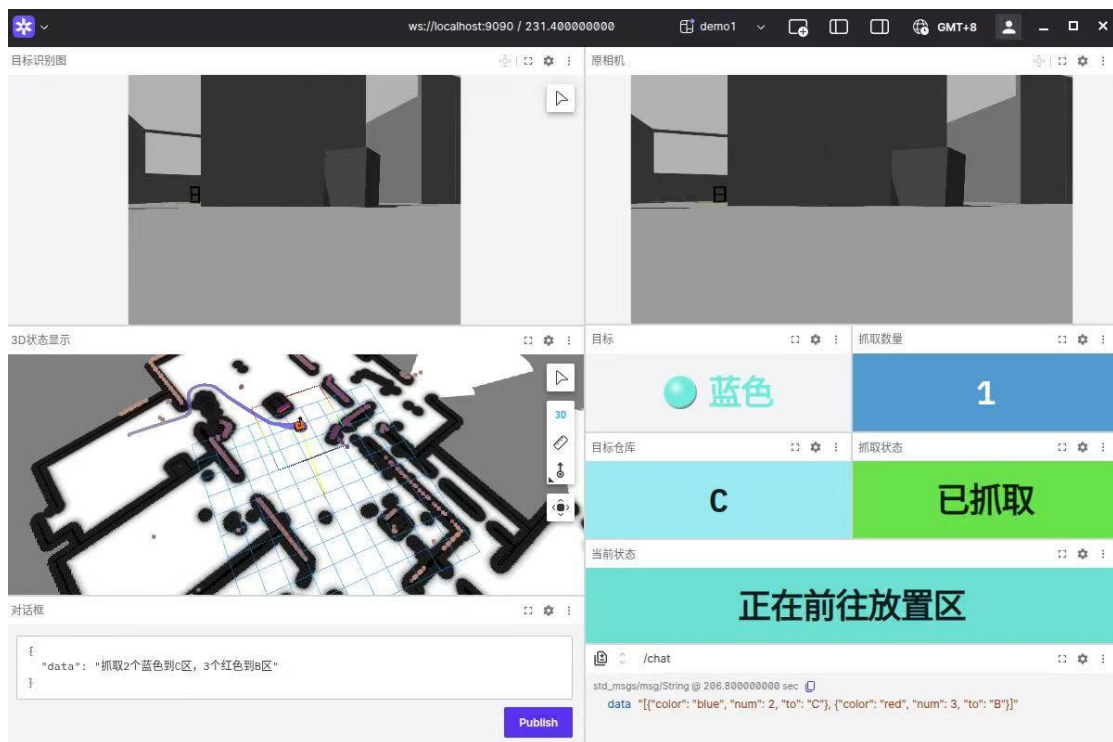


图 3-3 Foxglove 可视化界面图

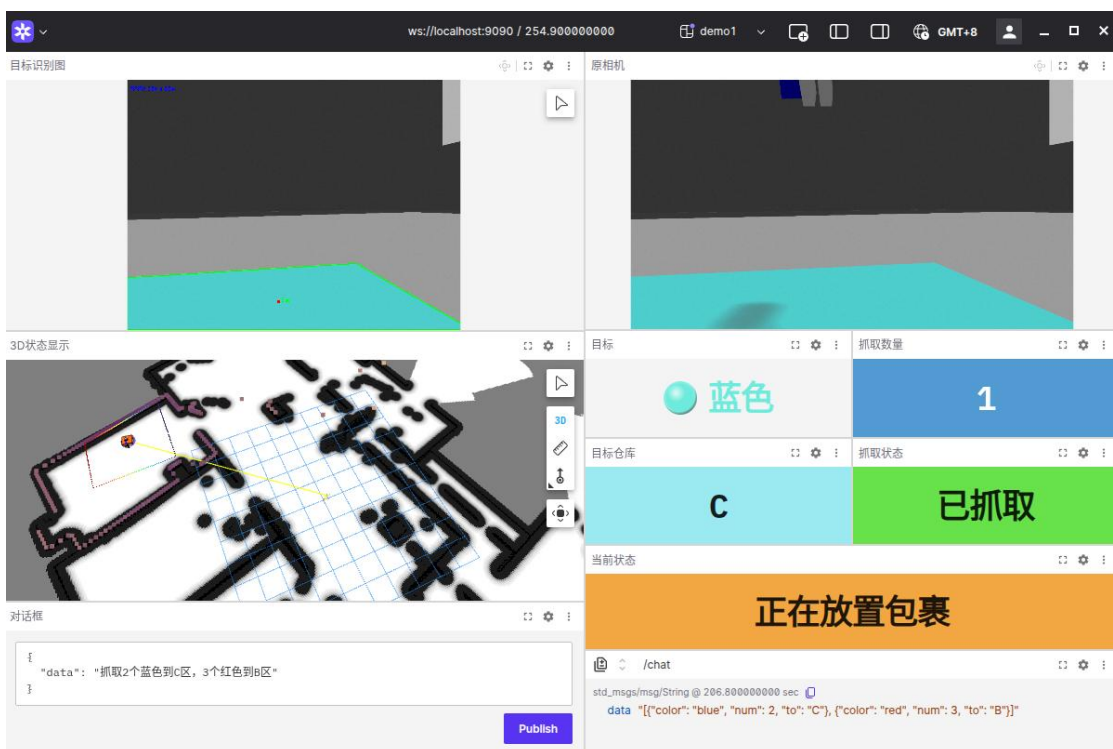


图 3-4 Foxglove 可视化界面图

3.2 系统集成与调试

3.2.1 模块间通信方式

具体如表 3-2

模块 1	模块 2	通信方式	具体接口	说明
LlamaChat	LlamaCommandParser	ROS Topic	llama_command	用户输入的自然语言命令经 LlamaChat 发布, 供命令解析器接收
LlamaCommandParser	MainController	ROS Topic	/chat	解析后的任务 JSON（包含 color、num、to 字段）发布到该话题, 供主控节点接收
MainController	nav_simple	ROS Topic	/manual_nav_target	主控节点发布导航目标位置（物块 / 放置区域）, 供导航模块执行

nav_simple	MainController	ROS Topic	/nav_statuses	导航模块发布当前导航状态（到达 / 未到达），供主控节点判断流程
MainController	arm_action_integration	ROS Topic	/current_target_cube	主控节点发布目标物块信息，供机械臂控制模块定位抓取位置
MainController	arm_action_integration	ROS Topic	/nav_done_cargo	导航到物块附近后，主控节点发布抓取触发信号
MainController	arm_action_integration	ROS Topic	/nav_done_area	导航到放置区域后，主控节点发布放置触发信号
arm_action_integration	MainController	ROS Topic	/arm_statuses	机械臂控制模块发布抓取 / 放置状态（成功 / 失败），供主控节点判断

arm_action_integrati on	IFRA_LinkAttacher	ROS Service	/ATTACHLIN K	机械臂控制模块调用吸附服务，实现物块与末端执行器的固定连接
arm_action_integrati on	IFRA_LinkAttacher	ROS Service	/DETACHLIN K	机械臂控制模块调用分离服务，解除物块与末端执行器的连接
相机传感器	package_detector	ROS Topic	/camera/im age_raw	相机发布原始图像数据，
雷达传感器	package_detector	ROS Topic	/scan	雷达发布激光点云数据，辅助物块检测模块定位
package_det ector	Foxglove 面板	ROS Topic	/detector/ annotated_ image	检测模块发布带标注的物块图像，供可视化面板显示
package_det ector	MainController	ROS Topic	（物块位置 数据，未明确命名）	检测模块发布识别到的物块，供主节点规划路径

AMCL 定位	MainController	ROS Topic	/amcl_pose	AMCL 发布机器人实时位姿，作为主控节点路径规划的基础
---------	----------------	-----------	------------	------------------------------

表 3-2 模块通信表

3.2.2 节点启动命令

一键启动脚本~/ros_competition_start.sh

该脚本是一款针对 ROS 2 机器人系统的一键自动化启动程序，基于 Bash 编写，核心作用是替代手动逐个启动终端、加载环境、执行指令的繁琐操作，脚本执行后会自动打开 10 个终端窗口，分别对应不同功能模块，终端内会输出各模块的启动日志；脚本末尾会提示“所有模块已启动”，用户只需检查各终端是否无报错（如“节点启动成功”“服务监听中”等），即可完成机器人系统的全流程启动，无需手动干预。具体见第四章图 4-3-1。

具体单个节点启动

1.启动仿真(机械臂+底盘+传感器)

```
ros2 launch mybot gazebo_world.launch.py
```

2.启动 moveit(机械臂规划、控制)

```
ros2 launch mybot my_moveit_rviz.launch.py
```

3.启动 nav2(导航)

```
ros2 launch bot_navigation nav_bringup_gazebo.launch.py
```

4.启动服务端

```
llama-server -m ~/llama.cpp/qwen2.5-coder-0.5b-instruct-q4_k_m.gguf -c 2048  
--threads 8 --port 8081
```

5.启动语言解析模块

```
ros2 run llama_command_parser command_parser
```

6.启动导航模块

```
ros2 run nav_simple simple_navigator
```

7.启动机械臂模块

```
ros2 run arm_action_integration arm_grab_place_node
```

8.启动视觉模块

```
ros2 run package_detector color_detector_node
```

9.启动主控模块

```
ros2 run main_controller main_controller_node
```

10.运行 rosbridge

```
ros2 launch rosbridge_server rosbridge_websocket_launch.xml
```

3.2.3 避障参数调整

```
linear_granularity: 0.05
angular_granularity: 0.025
transform_tolerance: 0.2
xy_goal_tolerance: 0.25
trans_stopped_velocity: 0.25
short_circuit_trajectory_evaluation: True
stateful: True
critics: ["RotateToGoal", "Oscillation", "BaseObstacle", "GoalAlign",
"PathAlign", "PathDist", "GoalDist"]
BaseObstacle.scale: 1.0
PathAlign.scale: 15.0
PathAlign.forward_point_distance: 0.1
GoalAlign.scale: 24.0
GoalAlign.forward_point_distance: 0.1
PathDist.scale: 32.0
GoalDist.scale: 24.0
RotateToGoal.scale: 60.0
RotateToGoal.slowing_factor: 1.0
RotateToGoal.lookahead_time: 0.3
```

图 3-5 导航部分配置文件

图上参数都是局部规划器的关键配置，这些参数直接影响机器人的局部路径规划

逻辑与运动控制行为，其调整的核心目标是提升导航的前瞻性、安全性与目标对准精度，具体影响分析如下：

局部轨迹仿真的时间范围是首个重要调整点，即 `sim_time`。该参数定义了 `DWBLocalPlanner` 仿真未来轨迹的时长，原始值 0.5 秒意味着规划器仅能基于极短的未来时间窗口生成轨迹，虽计算量小但对远处障碍物或路径趋势的预判能力薄弱，容易出现临近障碍才仓促调整路径的滞后问题。修改为 4.0 秒后，规划器的“视野”显著扩展，能提前 4 秒模拟运动轨迹，从而更早识别潜在障碍并规划平滑绕行路径，增强全局路径的协调性；不过，更长的仿真时间会增加实时计算负担，需确保硬件算力足以支撑，否则可能引发控制指令延迟。

避障优先级的提升是另一核心优化，体现在 `BaseObstacle.scale` 的调整上。`BaseObstacle` 作为避障评价器的权重参数，直接决定轨迹评分中避障因素的占比。原始值 0.3 下，避障优先级较低，机器人可能为贴合参考路径而过度靠近障碍物，存在碰撞风险；修改为 1.0 后，避障在轨迹评价中的权重显著提高，规划器会优先选择远离障碍的轨迹，安全性大幅增强，但代价是复杂障碍环境下路径可能更曲折——为规避障碍需牺牲部分路径平滑性。

针对目标朝向的控制，三个参数的联动调整进一步提升了精度与流畅性。其中，`RotateToGoal.scale` 提高了“朝向目标旋转”评价器的权重：原始值 32 对应的中等优先级可能导致机器人到达目标位置时朝向偏差较大（如角度误差超过阈值）；而 60.0 的高权重让机器人更主动地在接近目标时调整旋转角度，确保最终朝向与目标方向高度一致，尤其适配对接、定点作业等对朝向精度要求高的场景，但可能因优先调整朝向而降低前进速度，延长到达时间。

配合朝向权重的调整，调整 `RotateToGoal.slowing_factor` 从 6.0 到 1.0 减弱了旋转时的减速程度：原始值 6.0 意味着旋转时减速强烈，虽动作平稳但耗时较长，适合静态环境下的精细对准；修改为 1.0 后，减速作用大幅降低，旋转速度更快，能快速响应动态环境中目标方向的变化，但可能因减速不足导致旋转超调，需后续小幅修正。而 `RotateToGoal.lookahead_time` 则为旋转规划增加了前瞻机制：原始值 -1 表示

无前瞻，机器人可能临近目标时才急促旋转，动作生硬且易因惯性导致姿态不稳定；0.3 秒的前瞻时间让规划器提前预判旋转需求，在接近目标前便逐步调整朝向，避免“急转”，使旋转过程更连贯流畅。

整体来看，这些参数调整形成了一套协同优化策略：通过延长轨迹仿真时间增强路径前瞻性，提升避障权重保障安全，优化朝向控制参数兼顾精度与流畅性。这使得机器人更适应对导航精度（尤其是朝向）和复杂环境适应性要求较高的场景，如室内定点作业或动态障碍环境。不过，需注意 `sim_time` 增大带来的算力压力，以及 `BaseObstacle` 权重提高可能导致的路径曲折问题，实际应用中需结合硬件性能与场景需求进一步微调。

3.2.4 仿真物理环境鲁棒性优化

在系统集成测试与长时间的仿真运行中，团队发现由于 Gazebo 物理引擎（Physics Engine）在处理刚体碰撞时的非绝对真实性，系统存在极端的物理失稳风险。

1. 问题分析

在实际竞赛任务中，机器人需在狭窄的仓储环境中频繁移动。当机器人底盘或车轮边缘意外触碰或压到待抓取的立方体物块（Cargo Cube）时，物理引擎的瞬间穿透力计算容易产生异常的排斥力（类似于弹簧效应）。在原始设计中，由于机器人自重较轻且车轮半径较小，这种碰撞会导致机器人被瞬间“弹飞”、侧翻甚至剧烈抖动，直接导致定位丢失（AMCL 粒子发散）和任务失败。

2. 优化策略

为了解决这一物理仿真层面的鲁棒性问题，团队对机器人的 URDF/Xacro 物理描述文件进行了针对性的参数修正，主要采取了“增重稳固”与“结构越障”两大策略：

增加机身配重（Increase Mass & Inertia）：团队大幅提升了机器人底盘（Base Link）的质量（Mass）设定，并重新计算了对应的惯性矩阵（Inertia Matrix）。通过增加物

理自重，提高了机器人在仿真环境中的下压力和抓地力，使其在发生轻微碰撞或压到异物时，能够依靠惯性抵抗物理引擎的弹射力，保持姿态稳定。

增大车轮半径（Increase Wheel Radius）： 针对原始车轮容易被小方块卡住或在碾压时产生剧烈颠簸的问题，团队修改了车轮的碰撞几何参数，显著增大了左右驱动轮的半径。大半径车轮改变了机器人与障碍物接触的攻角（Angle of Attack），使得机器人在意外压到物块时，能够像越野车一样“跨越”或平稳地将物块推开，而不是被物块“绊倒”或顶飞。

3.优化结果对比

通过对比测试，优化后的机器人在通过障碍物或压过物块时，车身仅会有轻微起伏，随后迅速恢复平衡，未再出现翻车或弹飞现象，极大地提升了系统在 5 分钟连续作业中的容错率与鲁棒性。

为落实上述鲁棒性优化策略，我们在机器人底盘描述文件（URDF/Xacro）中对核心物理参数进行了重定义。以下代码片段展示了 `base_mass`（底盘质量）与 `wheel_radius`（车轮半径）的参数调整及惯性矩阵的应用逻辑：

```
01 <xacro:property name="base_mass" value="5.0" />
02
03 <xacro:property name="wheel_radius" value="0.2" />
04
05 <link name="base_link">
06   <visual>
07     ...
08   </visual>
09   <collision>
10     ...
11   </collision>
12   <xacro:box_inertia m="${base_mass}"
13     w="${base_width}" h="${base_height}"
14     d="${base_length}" />
15 </link>
```

图 3-6 底盘质量与车轮半径参数调整

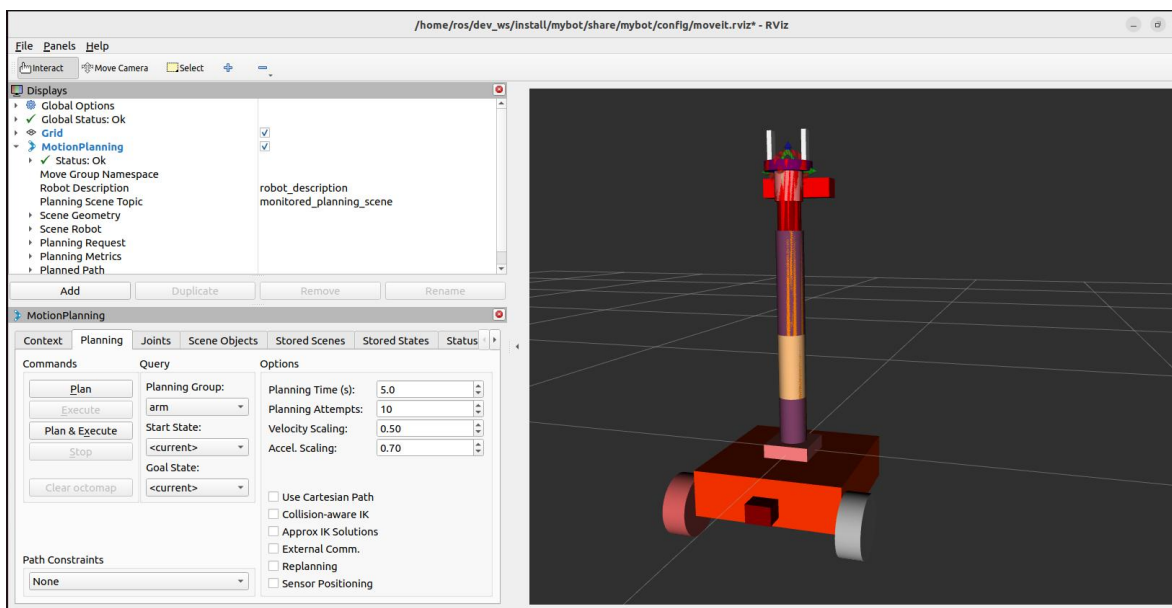


图 3-7 原始机器人模型（小轮径、轻量化，易因碰撞失稳）

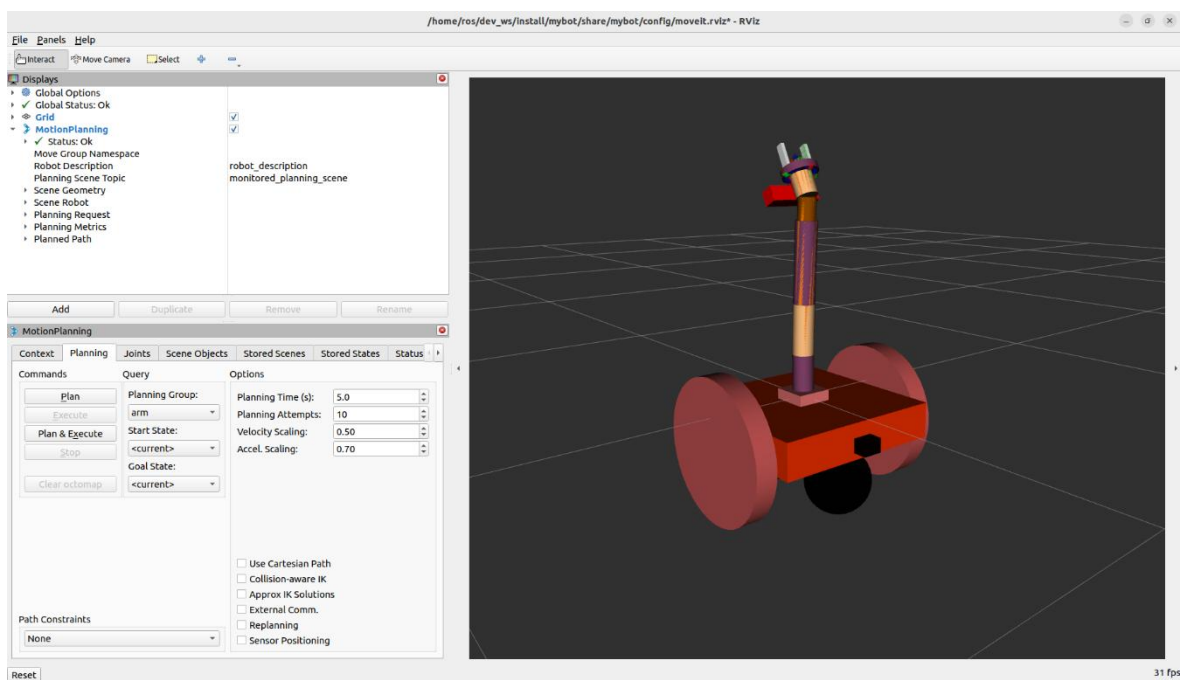


图 3-8 改进型机器人模型（大轮径、高自重，具备强抗干扰能力）

第四章 系统构建与导航节点解析

4.1 场景构建

4.1.1 基本场景构建

（1）确认赛事方提供的 ROS2、Gazebo、Nav2、MoveIt2、大模型均能正常运行，排除环境故障。

（2）更改物块与放置区域坐标，将红蓝色物块放置在房间的各个位置，充分测试导航功能，物块具体组表如表 4-1

红色物块(x,y)	蓝色物块位置
(7.6,5.5)	(8.4,-7.1)
(9.6,-3.7)	(5.0,-7.4)
(-5.7,5.5)	(-1.4,6.6)
(-8.7,1.0)	(-9.6,-3.2)
(-10.7,4.7)	(-3.7,0.8)

表 3-2 物块坐标表

4.1.2 移动障碍物

移动障碍物是通过 Gazebo 仿真环境中的模型插件实现自动移动的物体，主要用于模拟动态环境中的干扰因素，考验系统的避障与路径规划能力。

其核心运动逻辑基于预设路径的往返循环。每个障碍物配置了起点、终点，在两点间直接往返。运动过程中，障碍物会实时计算当前位置与目标点的距离，当距离小

于 0.1 单位时切换阶段，确保在预设路径上持续移动，移动速度通过参数固定（0.04 单位 / 仿真时间间隔）。

在 B 和 C 区域门口添加移动障碍物 Obstacle2 和 Obstacle3, 干扰小车的行进路线，测试小车的避障能力。

主控系统会实时监控移动障碍物的位置，通过定时器定期更新其坐标，并结合安全距离判断是否对机器人路径造成干扰。若检测到障碍物处于危险范围，系统会触发等待机制或重新规划路径，避免碰撞，确保机器人任务执行的安全性。

4.1.3 Gazebo 仿真结果

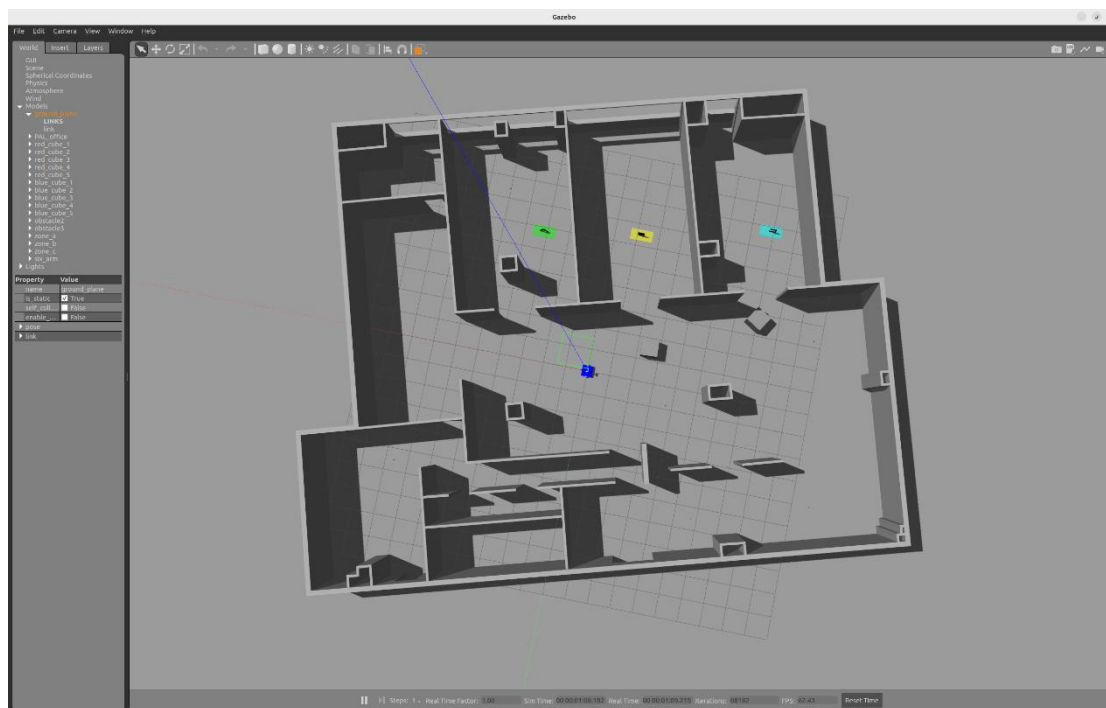


图 4-1 场景图

4.2 系统实现完整流程

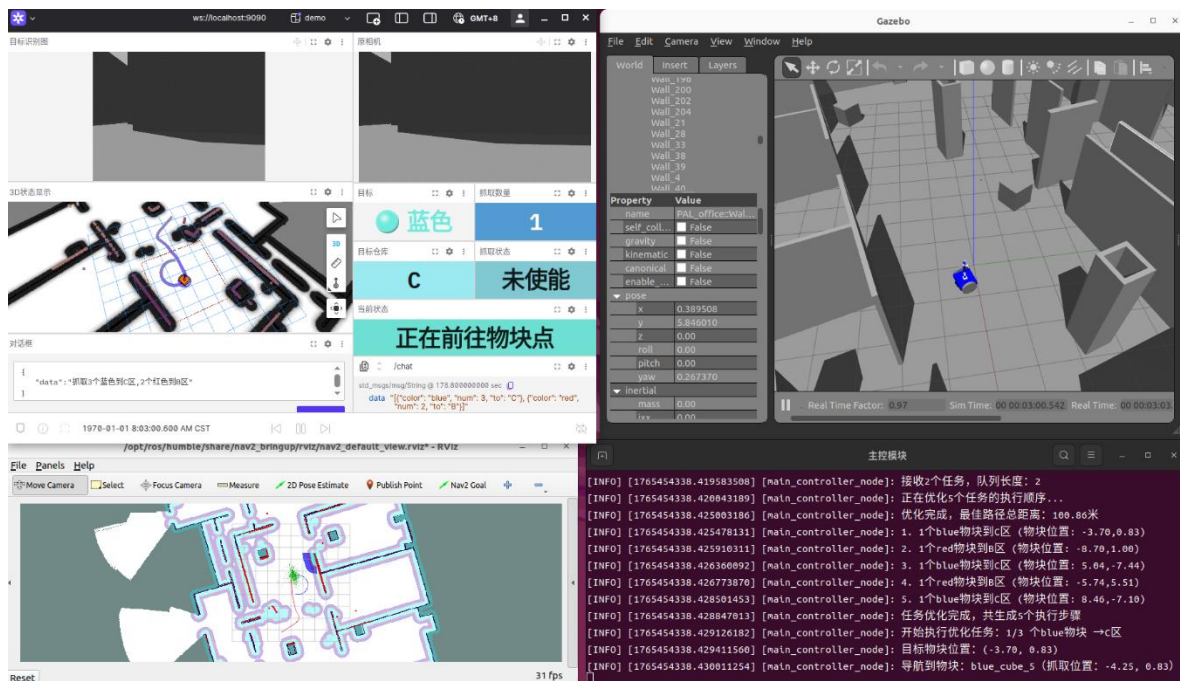


图 4-2-1 命令发布

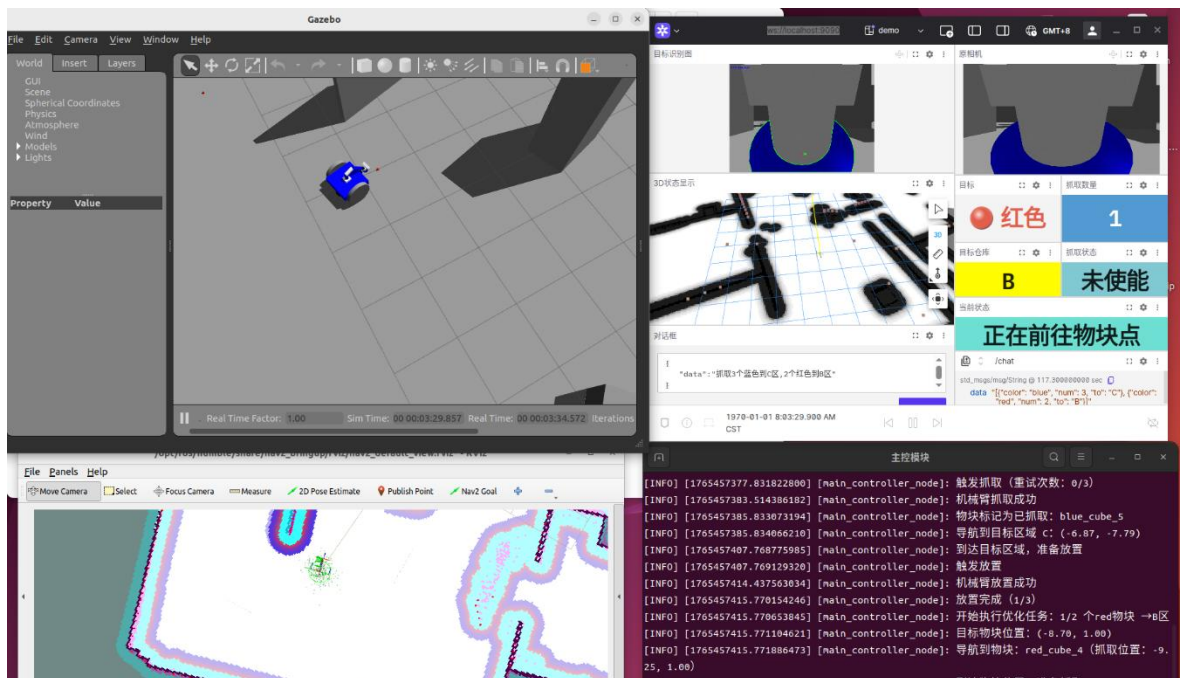


图 4-2-2 抓取物块

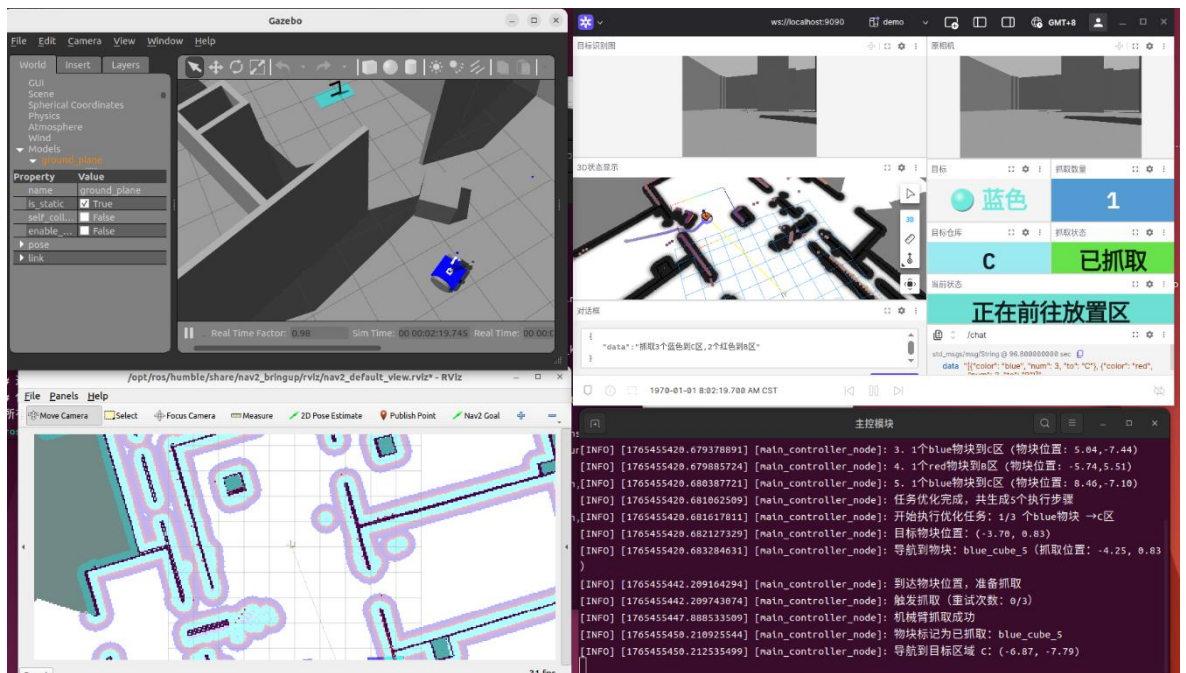


图 4-2-3 导航至放置区

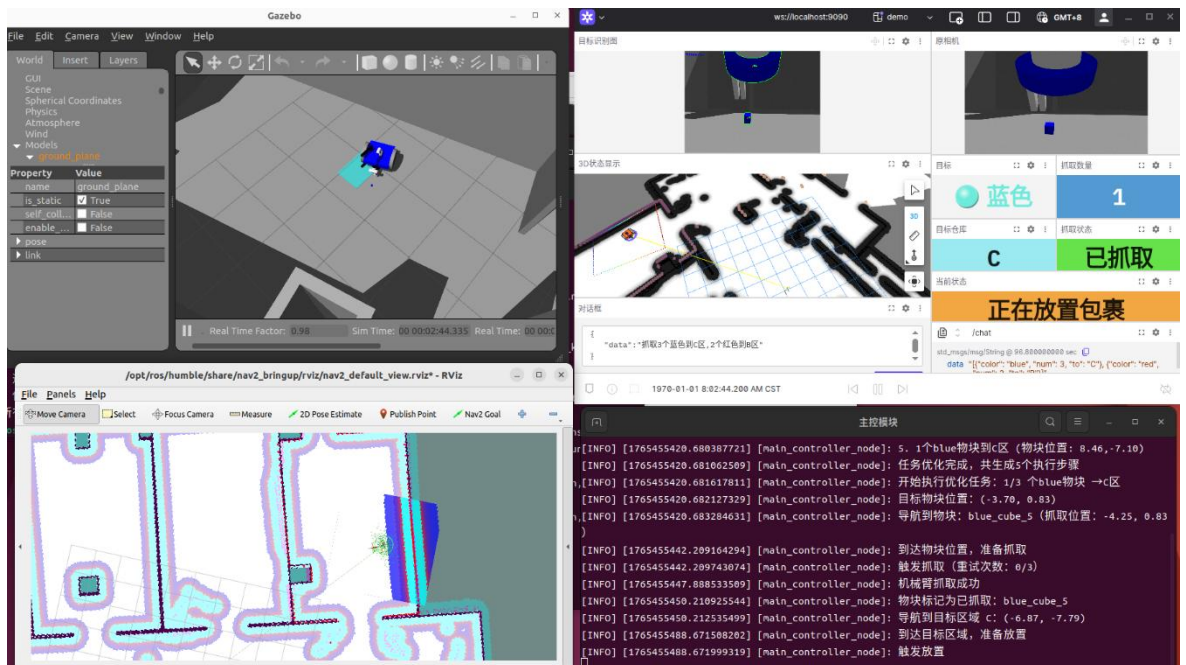


图 4-2-4 放置物块

4.3 核心代码展示

本系统核心算法位于 `MainControllerNode` 中，采用了基于 全排列 (Permutations) 的全局路径优化策略。不同于传统的贪心算法（仅考虑当前最近目标），本算法通过 `itertools.permutations` 生成所有可能的任务执行顺序，并结合动态物块分配机制，计算每种序列的“机器人→抓取点→放置区”总欧氏距离成本，从而筛选出全局最优执行方案。

以下是核心算法实现的详细代码：

1. 多任务序列全局优化算法

该函数是路径规划的大脑。它接收拆分后的原子任务列表，生成所有排列组合，并调用成本评估函数找到 `min_total_cost` 对应的最佳序列。

```
from itertools import permutations

def optimize_task_order(self, tasks):
    """
    基于全排列生成所有任务执行顺序，并筛选总路径成本最低的方案。
    解决多目标任务中的"旅行商问题"（TSP）变种。
    """
    # 单任务直接计算，无需排列
    if len(tasks) <= 1:
        cost, assignments = self.calculate_task_sequence_cost(tasks)
        return assignments

    self.get_logger().info(f"正在优化{len(tasks)}个任务的执行顺序...")

    min_total_cost = float("inf")
    best_assignments = None

    # 核心逻辑：全排列遍历所有可能的任务执行顺序
    # 例如任务 A, B -> 组合为 [A, B] 和 [B, A]
    for task_order in permutations(tasks):
        # 计算当前序列的总路径成本（包含抓取和放置的全程距离）
        cost, assignments = self.calculate_task_sequence_cost(task_order)

        # 贪心策略：保留当前发现的最小成本序列
```

```

        if cost < min_total_cost:
            min_total_cost = cost
            best_assignments = assignments

    if best_assignments:
        self.get_logger().info(f"优化完成，全局最优路径总距离：
{min_total_cost:.2f}米")
        return best_assignments

```

2. 序列成本评估与动态物块分配

此函数负责计算特定任务序列的“物理代价”。创新点在于 `assign_best_block_to_task`，它能在计算路径时，根据机器人当前的假设位置，动态选择“顺路”的物块，而非死板地指定物块 ID。

```

def calculate_task_sequence_cost(self, tasks):
    """
    预演特定任务序列的执行过程，计算总移动距离成本。
    """
    total_cost = 0.0
    current_pos = self.current_robot_pose # 模拟起始点为机器人当前位置
    used_blocks = set() # 记录已被规划占用的物块，避免冲突

    task_assignments = []

    # 1. 预演任务分配：为序列中的每个任务动态分配最优物块
    for task in tasks:
        # 在当前模拟位置下，寻找最近的同色可用物块
        best_block, cost = self.assign_best_block_to_task(task, used_blocks)

        if not best_block:
            return float("inf"), [] # 路径不可行（物块不足）

        # 解析关键坐标点
        block_x, block_y, block_idx = best_block
        target_area = task["to"]
        area_pos = self.AREA_COORDS[target_area]
        grasp_pos = self.get_grasp_position((block_x, block_y))

        # 记录分配结果供执行器使用
        task_assignments.append({

```

```

        "task": task,
        "block_pos": (block_x, block_y),
        "block_idx": block_idx,
        "grasp_pos": grasp_pos,
        "area_pos": area_pos
    })
    used_blocks.add(block_idx) # 标记该物块已被此方案占用

# 2. 计算累积路径成本
for assignment in task_assignments:
    # Cost += 当前位置 -> 抓取点距离
    total_cost += self.calculate_distance(current_pos,
assignment["grasp_pos"])
    # Cost += 抓取点 -> 放置区距离
    total_cost += self.calculate_distance(assignment["grasp_pos"],
assignment["area_pos"])
    # 更新模拟位置到放置区, 作为下一任务起点
    current_pos = assignment["area_pos"]

return total_cost, task_assignments

```

3. 大模型指令触发优化流程

展示如何将 LLM 解析出的非结构化指令（JSON）转化为优化流程的输入，体现了“大模型+运筹优化”的结合。

```

def _optimize_and_process_tasks(self):
    """
    接收大模型解析的原始指令，触发全排列优化流程。
    """
    if not self.task_queue:
        return

# 1. 任务原子化：将"抓取 2 个红方块"拆解为 2 个独立的单次抓取任务
expanded_tasks = []
for task in self.task_queue:
    num = task["num"]
    if num > 1:
        for i in range(num):
            subtask = task.copy()
            subtask["num"] = 1

```

```
        expanded_tasks.append(subtask)
    else:
        expanded_tasks.append(task)

    # 2. 调用核心算法获取最优执行队列
    # 输入: 乱序的原子任务列表
    # 输出: 路径最短的有序执行列表
    self.optimized_tasks = self.optimize_task_order(expanded_tasks)

    if self.optimized_tasks:
        self.get_logger().info(f"生成最优策略: 共{len(self.optimized_tasks)}步,
立即开始执行...")
        self._process_next_optimized_task()
```

参考文献

- [1] Macenski S, Foote T, Gerkey B, et al. Robot Operating System 2: Design, architecture, and uses in the wild[J]. Science Robotics, 2022, 7(66): eabm6074.
- [2] Macenski S, Xiao T, Lim D, et al. Marathon 2: A navigation system[C]//2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2020: 2718-2725.
- [3] Touvron H, Lavril T, Izacard G, et al. LLaMA: Open and Efficient Foundation Language Models[J]. arXiv preprint arXiv:2302.13971, 2023.
- [4] Chitta S, Ioan S, Piccoli M. MoveIt!: An open source robotics manipulation platform[J]. Robotics & Automation Magazine, IEEE, 2016, 21.
- [5] Bradski G. The OpenCV Library[J]. Dr. Dobb's Journal of Software Tools, 2000, 25(11): 120-123.
- [6] Koenig N, Howard A. Design and use of architectures for the simulator Gazebo[C]//2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE, 2004, 3: 2149-2154.
- [7] Vemprala S, Bonatti R, Bucker A, et al. ChatGPT for Robotics: Design Principles and Model Abilities[J]. IEEE Access, 2024, 12: 55682-55696.
- [8] Dantzig G B, Fulkerson D R, Johnson S M. Solution of a large-scale traveling-salesman problem[J]. Operations Research, 1954, 2(4): 393-410.